

Exam — What To Expect (SB5-MAI)

What the exam actually is, what gets graded, the answer pattern the examiner wants, what you may bring, and how to use this Lookup tool on the day. Format claims are cited to the student-notes summary `What-To-Expect.pdf` as (What-To-Expect p.N); see the Provenance note at the end for how much weight to give them.

Exam format — what happens on the day

A reflective report, written at the exam

The exam is **not** a conventional question-sheet exam. Its core is a **reflective report about the lab work you did during the course**, combined with some theory questions drawn from lectures, slides, and the textbooks (What-To-Expect p.1). You write the report **during the exam**: at the start you receive a **report template** containing different report sections, reflection questions, and questions related to your own lab work and code (What-To-Expect p.1). Your job on the day is to fill that template with well-argued prose.

The report should be based on three things you already own before you walk in (What-To-Expect p.1):

- **Your GitHub repository** — the fork of JHotDraw you worked on all semester.
- **The source code you worked on** — the actual classes you changed, refactored, and tested.
- **The exercises and modifications you made during the course** — the lab portfolio work.

The mix: mostly reflection, some theory

Most of the report is reflection on the practical work — *how you did the work* — but there will also be some questions about course-material theory from the slides or textbook "to have more examination" (What-To-Expect p.6). Expect the theory questions to be short and definitional (the two named examples are "How do you identify a code smell?" and "What is meant by a refactoring pattern?" — What-To-Expect p.3, p.6), while the reflection questions ask you to narrate and justify your own changes.

What the examiner cares about most

The lecturer was explicit that the exam is not only about *what* you did, but especially (What-To-Expect p.1):

- **Why you did it.**
- **How you did it.**

Every section of your report should connect your practical work to software-maintenance concepts and clean-code principles (What-To-Expect p.1). A bare description of an action earns little; an action plus its reasoning plus the concept it instantiates is what the format is designed to extract. The whole answer pattern is unpacked in the why/how section below.

What gets graded (report + repository)

The report

The report you produce at the exam is the primary deliverable: a filled template whose sections reflect on your lab work, explain how you implemented things, explain why you made certain technical decisions, and connect the practice to maintenance and Clean Code concepts (What-To-Expect p.1). The report should **directly reference your repository and source code** (What-To-Expect p.5) — name real classes, real commits, real pipeline files. Generic essays score worse than specific ones anchored in your own diff.

The repository

The **GitHub repository itself will also be evaluated** (What-To-Expect p.5–6). The grade includes the repository work — "what we did with the code" (What-To-Expect p.6). Specifically listed as evaluated (What-To-Expect p.5):

- **Code changes** — the actual feature/change work from the labs.
- **Refactoring work** — smells found and refactorings applied (the RefactLab feature branch).
- **Testing** — JUnit/AssertJ unit tests and JGiven scenarios committed to the repo.
- **Pipeline setup** — the GitHub Actions workflow under `.github/workflows/` (CILab p.1).
- **Overall maintenance work** — commit history, branches, issues, the general signs of disciplined change.

How report and repository connect

Treat the two as one artifact: the repository is the **evidence**, the report is the **argument**. Each claim in the report ("I removed a Long Method smell from class X by Extract Method") should be checkable against a commit in the repository. Conversely, every significant piece of repository work (a refactoring branch, a workflow file, a test class) should be claimed and justified somewhere in the report — unexplained work earns less than explained work, because the explanation is what the exam measures (What-To-Expect p.1, p.5).

Practical implication for the days before the exam: tidy the repository now. Make sure the pipeline is green, the README orients a reader, the feature branches are pushed, and your commit messages tell the story you want the report to tell.

The why/how answer pattern

The rule: decision → rejected alternative → concrete benefits

The lecturer wants **justification behind decisions** (What-To-Expect p.4). The strongest sentence shape, used throughout the model answers in this study package, has three parts:

1. **The decision** — what you did ("I used a switch statement...").
2. **The rejected alternative** — what you did *not* do, named explicitly ("...instead of multiple if-statements...").
3. **The concrete benefits** — why your choice wins, in maintenance vocabulary ("...because it improved readability and provided clearer structure for handling multiple conditions.").

The lecturer's own exemplar (switch statement)

Verbatim from the summary (What-To-Expect p.4):

- **Bad:** "I used a switch statement."
- **Good:** "I used a switch statement instead of multiple if-statements because it improved readability and provided clearer structure for handling multiple conditions."

The bad version is a fact; the good version is an argument. Nearly every reflection question in the template can be answered by stacking three to five sentences of this shape.

More good/bad pairs (constructed on the same pattern)

The following pairs are constructed for this guide on the lecturer's pattern, grounded in the course material:

- **Bad:** "I extracted a method." **Good:** "I extracted the duplicated update logic into a single named method instead of leaving the copies in place, because duplicated code means every future fix must be made in several places and one is inevitably missed — Extract Method gave the logic one home and a name that documents its intent (Refactoring1 p.6, p.11)."
- **Bad:** "I wrote unit tests." **Good:** "I wrote JUnit unit tests instead of relying on manual checks, because unit tests are fast and focused — no database or file system — and an automated suite re-runs on every change, which is what made my later refactoring safe to do (ContinuousIntegration p.9; Software Testing p.81)."
- **Bad:** "I used a feature branch." **Good:** "I developed on a feature branch cut from `development` instead of committing directly to the mainline, because the branch isolated my work-in-progress from the team baseline and let the pull request trigger the CI build before integration (RefactLab p.1; CILab p.1)."
- **Bad:** "I added an interface." **Good:** "I introduced an interface between the client and the concrete class instead of letting the client construct the class directly, because depending on an abstraction (DIP) lets me add new implementations without editing tested code (OCP), so the change ripple stops at the interface (OOPrinciples p.6, p.12)."
- **Bad:** "My pipeline builds with Maven." **Good:** "I configured the GitHub Actions workflow to build every pull request with Maven and execute the tests automatically, rather than building manually before hand-ins, because building every commit shortens the time between defect introduction and detection and makes the build server — not my laptop — the final authority on whether the code works (ContinuousIntegration p.6–7; CILab p.1)."
- **Bad:** "I renamed some variables." **Good:** "I renamed the cryptic locals to intention-revealing names instead of adding explanatory comments, because a good name removes the decoding cost permanently while a comment drifts out of date — the code now explains itself (CleanCode p.11; Refactoring1 p.8)."

A fill-in skeleton you can reuse for any decision

"I chose **[decision]** instead of **[alternative]** because **[benefit 1 in maintenance vocabulary]** and **[benefit 2]**. This follows **[named principle/refactoring/practice]**, which matters here because **[link to maintainability, change cost, or verification]**."

If you can fill that skeleton honestly for a decision in your repository, you have an exam-grade paragraph.

Benefit vocabulary — the words that carry the "why"

The third slot of the pattern (concrete benefits) is where marks are won, and it works best in the course's own vocabulary. Keep this word bank open while writing; every term below is exam-legitimate because it comes from the course material:

- **Readability / understandability** — the code is cheaper for the next reader to decode; refactoring's stated goal (Refactoring1 p.3).
- **Cheaper to modify / maintainability** — the next change costs less; the other half of the refactoring definition (Refactoring1 p.3).
- **Behaviour preservation** — the change is provably safe because observable behaviour is unchanged and tests verify it (Refactoring1 p.3).
- **Smaller impact set / shorter change propagation** — fewer classes are touched by future changes; the true cost lever of software change (ImpactAnalysis p.3–4; Drawlets p.37).
- **Open for extension, closed for modification** — new variants are added without editing tested code (OOPrinciples p.6).
- **Low coupling / high cohesion** — fewer reasons for change to ripple; each class has one job (OOPrinciples p.21–22).
- **Testability** — the unit can be verified in isolation, fast, without a database or GUI (ContinuousIntegration p.9; Software Testing p.29).
- **Regression safety** — the automated suite catches newly broken behaviour within minutes of the change (Software Testing p.81).
- **Shorter defect-introduction-to-removal gap** — the pipeline finds problems while the diff is small and fresh (ContinuousIntegration p.7).
- **Paying down technical debt / reducing interest** — less recurring cost every time this code is visited (BeyondTechnicalDebt p.12).

One benefit term, correctly attached to a real decision, beats three vague adjectives like "better", "cleaner", or "nicer" — none of which name a mechanism.

Allowed aids and the AI ban

What you may bring

The exam is effectively open-material: "we can basically prepare a lot beforehand" (What-To-Expect p.2). Allowed, per the lecturer (What-To-Expect p.2):

- **Textbooks**
- **Portfolio material** (your lab hand-ins)
- **Lecture notes** and personal notes/files
- **Prewritten templates/drafts** — including a prewritten report
- **Diagrams** — prepared in advance
- **Downloaded materials** — "anything that can be downloaded... anything you want" (What-To-Expect p.6)
- **Internet access (possibly)** — the lecturer himself was not certain internet would be available, and recommended relying on downloaded/local material "just in case internet access becomes limited" (What-To-Expect p.2, p.6). Treat internet as a bonus, not a plan.

Copying from your own prepared material is explicitly sanctioned: "you can copy paste some stuff from prewritten report" and rewrite/adapt prepared reflections during the exam (What-To-Expect p.2, p.6).

What is banned: generative AI

The single named prohibition is **generative AI tools** (What-To-Expect p.2). The lecturer specifically said to **disable AI features/modes in browsers and search engines before the exam** (What-To-Expect p.2, p.6). Concretely, before the exam:

- Turn off AI answers/AI mode in your search engine and browser sidebar assistants.
- Close or sign out of any chat-assistant tools; do not have them on the machine you bring.
- Make this offline study package (Lookup + guides) your primary reference — it is retrieval over your own prepared notes, not a generative tool, which is exactly the kind of downloaded/local material the lecturer recommends (What-To-Expect p.2).

Why local material beats the internet

Two reasons. First, internet availability is uncertain (What-To-Expect p.2). Second, time: at the exam you do not want to be searching the open web for a definition you could have had pre-written and indexed. The preparation strategy below exists to make the exam a *retrieval-and-adapt* exercise, not a *research* exercise.

The lecturer's preparation checklist

Recommended preparation (the lecturer's own list)

The lecturer "strongly hinted that we can prepare a lot in advance" and recommended (What-To-Expect p.2):

1. **Create a prewritten report template** — write the report about every lecture's content in advance, "every single thing", and bring it to rewrite or reuse (What-To-Expect p.6).
2. **Write notes for every lecture topic** — one prepared explanation per topic.
3. **Prepare explanations for all exercises/labs** — in the prewritten report, "write about exercises and how you have modified the code" (What-To-Expect p.7).
4. **Prepare diagrams beforehand** — e.g. the clean-architecture circles, the eight-phase change-process diagram, your pipeline's stage diagram (What-To-Expect p.2, p.4).
5. **Prepare reflections on code modifications** — the why/how paragraphs for each change you made, ready to copy and adapt (What-To-Expect p.2).

This study package implements that checklist: the lecture guides are the per-topic notes; the companion file `91-exam-model-answers.md` is the prewritten reflection bank.

The seven emphasized topic areas

The summary names these as the important topics to prepare (What-To-Expect p.3–5):

1. **Refactoring** — how you refactored, why, and which techniques/patterns you used (What-To-Expect p.3). Sample question: "*What is meant by a refactoring pattern?*"
2. **Code smells** — how to identify them, examples of bad structure, why certain code is problematic (What-To-Expect p.3). Sample question: "*How do you identify a code smell?*"
3. **Software testing** — how you applied it, why testing was important, what kinds of tests you used (What-To-Expect p.3).

4. **Continuous delivery & pipelines** — flagged as "an important topic"; the lecturer specifically mentioned continuous delivery, pipelines, and CI/CD-related work, and a past example report ended with a pipeline question (What-To-Expect p.3–4, p.6). Sample questions: *"How did you create your pipeline?"*, *"How does the pipeline work?"*, *"Why did you structure it that way?"*
5. **Technical decision-making** — the why/how justification pattern above (What-To-Expect p.4).
6. **Clean Code & architecture** — concepts from the Clean Code book, clean-architecture concepts, the diagrams on Itslearning; apply the concepts to your own code, explain design decisions, explain how you improved maintainability/readability (What-To-Expect p.4–5).
7. **GitHub repository quality** — code changes, refactoring, testing, pipeline setup, overall maintenance work (What-To-Expect p.5).

Mapping the seven areas to this study package

- Refactoring and code smells → `lecture-04-refactoring-maintainable-code.md`
- Testing → `lecture-07-software-testing.md`; BDD/JGiven verification → `lecture-09-bdd-verification.md`
- Continuous delivery & pipelines → `lecture-03-impact-analysis-processes-ci.md` (CI section and CILab)
- Technical decision-making → the why/how pattern above plus every "Why & how" block in `91-exam-model-answers.md`
- Clean Code & architecture → `lecture-06-clean-code-design-patterns.md` and `lecture-05-actualization-clean-architecture.md`
- Repository quality → `lecture-01-introduction-version-control.md` (Git areas, GitHub flow, Issues)
- The full change process end-to-end → `lecture-10-conclusion-worked-example.md` (Drawlets walkthrough)

Exam-day copy-paste workflow

The three-step loop

This Lookup tool is designed to be used *during* the exam (it is local, offline, and retrieval-only — no generative AI involved). For each question in the report template:

1. **Search the question verbatim.** Paste the template's question into Lookup exactly as written. The model-answer file (`91-exam-model-answers.md`) keys its sections on expected exam questions, so a verbatim or near-verbatim question should surface a section titled with that question (results appear as "Question › Copy-paste answer").
2. **Copy the answer block.** Open the matching "Copy-paste answer" section and copy it into the report template. It is written in the first person, in the decision → alternative → benefit pattern, and is self-contained.
3. **Personalize the placeholders.** Check the section's "Adapt to your lab" bullets: they list exactly what to swap in — your actual class names, the smell you actually found, your actual pipeline stages. Replace every bracketed placeholder such as `[the class you refactored]` with your real artifact names, and adjust any claim that does not match what your repository actually shows.

Making the answer yours

- **Anchor in your repo.** Add one sentence naming the concrete commit/branch/file: "...as can be seen in the `feature/remove-long-method` branch." The report should directly reference your repository (What-

To-Expect p.5).

- **Strip or keep citations.** The model answers carry slide citations like (Refactoring1 p.3). Keeping them shows you know the source material; stripping them is also fine — the report is reflection, not an academic paper. Be consistent.
- **Stack add-ons.** Each model-answer section has a "Why & how — justification add-ons" block of extra standalone sentences. If a question feels thin after the main answer, append one or two add-ons that are true for your work.
- **Never paste something false.** If the model answer claims a refactoring you did not perform, swap in one you did (the "Adapt to your lab" bullets tell you which part is swappable). A justified true sentence beats an impressive false one — the repository is also graded, and it will contradict you (What-To-Expect p.5).

A worked example of the loop

Suppose the template asks: *"How did you refactor the code and why?"*

1. **Search** Lookup for the question text. The top hits should include "How and why did you refactor your code, and which refactoring techniques did you use? › Copy-paste answer" from the model-answers file, plus supporting theory sections from the Lecture 4 guide.
2. **Paste** the copy-paste answer into the template section. As written it claims: feature branch from `development`, SonarLint findings, Extract Method / Move Method / Extract Class / Replace Conditional with Polymorphism, tests after each step.
3. **Personalize** against your repository. Say your actual work was one Extract Method on a long tool method plus a rename pass: delete the techniques you did not use, keep Extract Method, and replace the placeholder-level claims with your branch name and class name. Add one anchoring sentence: "The full sequence is visible on the `[your-branch]` branch, commits `[first]` – `[last]`."
4. **Stack one add-on** from the "Why & how" block that is true for you — e.g. the sentence about choosing Extract Method over an explanatory comment — so the answer ends on a justification.
5. **Sanity-check:** every sentence now true of your repo? Every decision paired with a rejected alternative and a benefit? Move on.

Total time per question with prepared material: a few minutes. That is the entire point of preparing this package.

Time strategy

Answer the reflection questions first (they are most of the grade and you have prepared material for them — What-To-Expect p.6), then the short theory questions (search the exact phrase; the lecture guides' "Definitions & Terminology" sections give one-line definitions). Reserve the last block of time to re-read the whole report once, checking every paragraph has a *why*, not just a *what* (What-To-Expect p.1).

Provenance note

What this document is based on

Everything cited here as (What-To-Expect p.N) comes from a **second-hand, student-authored summary**: a 7-page PDF whose pages 1–5 are a structured write-up "based on what the lecturer explained" and whose pages 6–7 are the student's own raw lecture notes ("Brain dump from my notes...", What-To-Expect p.6). It is **not** an official exam regulation, course description, or examiner document.

Confidence levels

- **High confidence** (stated consistently in both the summary and the raw notes): reflective report from a template handed out at the exam; based on repo + source code + exercises; repository itself graded; why/how emphasis; generative AI banned; prewritten material and copy-paste allowed; refactoring, code smells, testing, and pipelines as named topics; the five sample questions (What-To-Expect p.1–6).
- **Soft claims** (flagged as uncertain in the source itself): **internet access** — the notes say the lecturer "doesn't know if we can or cannot use the internet" (What-To-Expect p.6), so plan for offline; the exact template structure and section list are unknown until the exam starts; "possibly" qualifiers in the allowed-aids list are the source's own (What-To-Expect p.2).
- **Unverified extrapolations**: the derived question list in `91-exam-model-answers.md` beyond the five verbatim samples is this study package's prediction from the named topic areas, not a leaked question set.

What to do about the uncertainty

Check Itslearning and any official exam-information page for the authoritative rules (duration, aids, submission format) before the exam, and let official information override this document wherever they conflict. The preparation strategy here — prewritten reflections, local materials, why/how pattern — is robust either way: it helps under any plausible variant of the format.